8-bit MIPS nRisc

Kayky Moreira Praxedes, Carlos Ernesto Cardoso dos Reis

March 2026

1 Project Objective

Construction and simulation of a **single-cycle *MIPS nRisc* processor**, designed entirely in Verilog HDL, with a simplified 8-bit architecture for data, instructions, and memory [01].

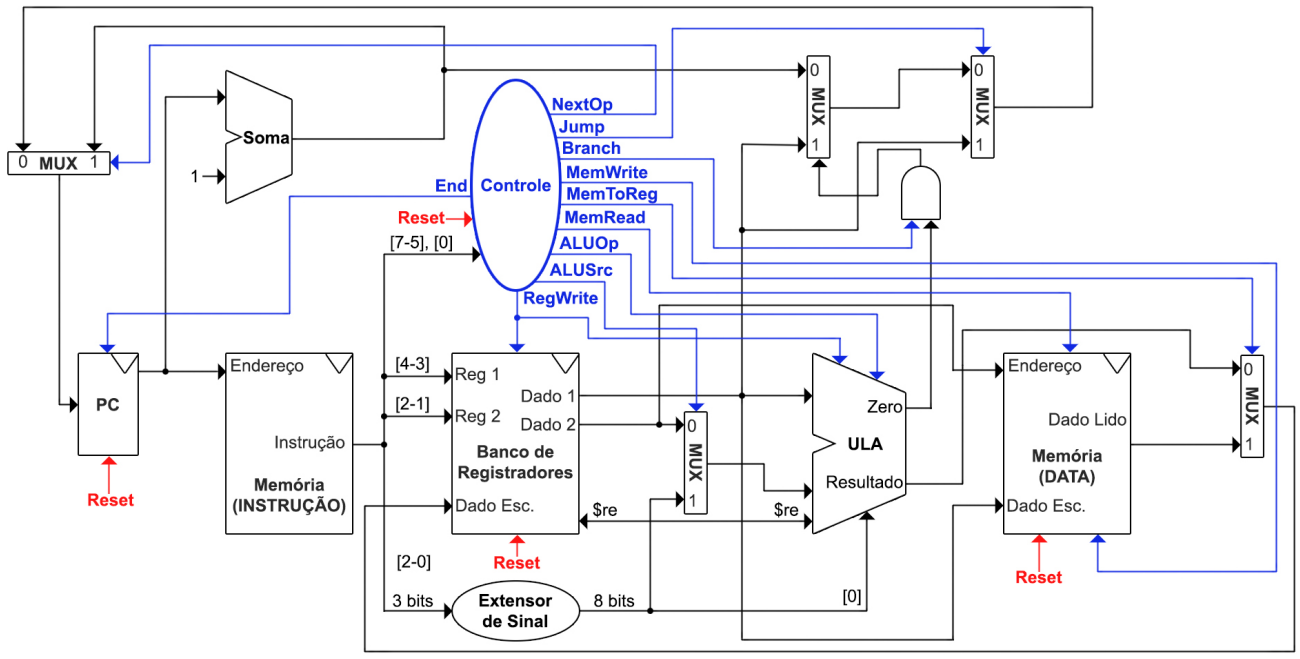
Embedded program:

A program that performs the Caesar Cipher was embedded in the instruction memory, operating on a message stored in data memory (see Appendix C). The algorithm:

1. Reads a character from memory.
2. Adds the shift (key, a number between -15 and 15).
3. Adjusts the value if it exceeds the alphabetic range.
4. Stores the new character back into memory.

The message limit is 128 characters, imposed by the address space (the same applies to the program size).

2 Diagram



Datapath:

The data follows a fixed order:

1. *PC* accesses instruction memory
2. Instruction is decoded by the control unit
3. Registers are read
4. ALU performs the operation
5. Result is written to the register file or memory

3 Architecture

What the main components are and how they work (see Appendix B).

1 Control Unit:

Generation of control signals from the *OpCode* and the *funct* field (they determine the datapath flow and the operation performed in each cycle).

Control *Flags* (binary):

	0000 add	0001 sub	001X addi	0100 mult	0101 null	0110 ld	0111 st	1000 jr	1001 null	1010 beq	1011 slt	1100 result	1101 null	1110 nop	1111 hlt
ALUSrc	0	0	1	0	0	0	0	0	0	0	0	1	1	1	1
MemToReg	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
RegWrite	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0
MemRead	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0
MemWrite	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
Branch	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
Jump	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
NextOp	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
End	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
ALUOp	00	00	01	10	10	01	01	01	01	11	11	00	00	01	01

- `ALUSrc`: Indicates whether the second operand comes from a register (0), or is an immediate value (1).
- `MemToReg`: Indicates whether the value to be written to the register comes from the ALU (0) or from memory (1).
- `RegWrite`: Indicates whether there will be a write to the register (register 1) (1) or not (0).
- `MemRead`: Indicates whether there will be a memory read (1) or not (0).
- `MemWrite`: Indicates whether there will be a memory write (1) or not (0).
- `Branch`: Indicates whether there will be a conditional branch (1), or not (0)
- `Jump`: Indicates whether there will be an unconditional jump (1), or not (0)
- `NextOp`: Ignores the instruction line.
- `End`: Ends the program.
- `ALUOp`: 2 *bits* indicating which type of operation will be performed by the ALU $_{[02]}$:

- 00: `add`, `sub`, or `result`
- 01: `addi`
- 10: `mult`
- 11: `beq` or `slt`

2 *PC (program counter)*:

Register responsible for storing the address of the next instruction.

3 Memory:

Memory is divided into two regions, both with 7-*bit* addresses (128 positions) and storing 8-*bit* data, namely:

- **Data memory:** Responsible for storing variable data (in this case, the message to be encoded).
- **Instruction memory:** Responsible for storing the program code.

4 Register File:

A set of 4 general-purpose registers with 8 *bits* each, accessed by instructions through a numeric ordering.

- 00: $\texttt{\$c}_0$
- 01: $\texttt{\$c}_1$
- 10: $\texttt{\$c}_2$
- 11: $\texttt{\$c}_3$

There is also the special register `$re`, responsible for storing the result of comparison instructions (`beq` and `slt`). The values it can take are 0 (comparison is false) or 1 (true), defaulting to -4 (11111100), which avoids unintended branches.

5 ALU (Arithmetic Logic Unit):

Responsible for arithmetic, logic, and comparison operations [03].

The project does not use behavioral description (high-level arithmetic operators of the language, such as `+`, `-`, `*`, or relational operators) **for the operations**. Instead, the ALU was built hierarchically, from basic components (1-bit adder, 8- and 16-bit adders), making the *carry* flow and signal propagation explicit, which allows the physical implementation of the circuit to be observed directly.

Sign extender:

Small module responsible for converting the 3-bit immediates from instructions to 8 *bits* (so that operations can be performed in the ALU).

4 Instructions (binary and Assembly)

Instructions are 8 *bits* long and are divided into three formats, depending on the number of registers used in the operation [04].

Type 2R:

Operations between two registers.

$$\underbrace{OpCode}_{3\ bits}\ \underbrace{Reg1}_{2\ bits}\ \underbrace{Reg2}_{2\ bits}\ \underbrace{funct}_{1\ bit}$$

Arithmetic:

- 000 00-11 00-11 0 ⇒ `add` $\texttt{\$c}_\texttt{i}$ $\texttt{\$c}_\texttt{j}$ ⇒ $\texttt{\$c}_\texttt{i} = \texttt{\$c}_\texttt{i} + \texttt{\$c}_\texttt{j}$
- 000 00-11 00-11 1 ⇒ `sub` $\texttt{\$c}_\texttt{i}$ $\texttt{\$c}_\texttt{j}$ ⇒ $\texttt{\$c}_\texttt{i} = \texttt{\$c}_\texttt{i} - \texttt{\$c}_\texttt{j}$
- 010 00-11 00-11 0 ⇒ `mult` $\texttt{\$c}_\texttt{i}$ $\texttt{\$c}_\texttt{j}$ ⇒ $\texttt{\$c}_\texttt{i} = \texttt{\$c}_\texttt{i} \times \texttt{\$c}_\texttt{j}$

Memory:

- 011 00-11 00-11 0 ⇒ `ld` $\texttt{\$c}_\texttt{i}$ $\texttt{\$c}_\texttt{j}$ ⇒ $\texttt{\$c}_\texttt{i} = \texttt{data\_memory[\$c}_\texttt{j}\texttt{]}$
- 011 00-11 00-11 1 ⇒ `st` $\texttt{\$c}_\texttt{i}$ $\texttt{\$c}_\texttt{j}$ ⇒ $\texttt{data\_memory[\$c}_\texttt{j}\texttt{]} = \texttt{\$c}_\texttt{i}$

Comparison:

- 101 00-11 00-11 0 ⇒ `beq` $\texttt{\$c}_\texttt{i}$ $\texttt{\$c}_\texttt{j}$ ⇒ $\texttt{\$re} = (\texttt{\$c}_\texttt{i} == \texttt{\$c}_\texttt{j})\ ?\ 1 : 0$
- 101 00-11 00-11 1 ⇒ `slt` $\texttt{\$c}_\texttt{i}$ $\texttt{\$c}_\texttt{j}$ ⇒ $\texttt{\$re} = (\texttt{\$c}_\texttt{i} < \texttt{\$c}_\texttt{j})\ ?\ 1 : 0$

Type 1R:

Operations with a single register.

$$\underbrace{OpCode}_{3\ bits}\ \underbrace{Reg}_{2\ bits}\ \underbrace{Immediate}_{3\ bits}$$

Arithmetic:

- 001 00-11 000-111 ⇒ `addi` $\texttt{\$c}_\texttt{i}$ *immediate* ⇒ $\texttt{\$c}_\texttt{i} = \texttt{\$c}_\texttt{i} + immediate$

Branch:

- 100 00-11 000-111 ⇒ `jr` $\texttt{\$c}_\texttt{i}$ ⇒ $\texttt{PC} = \texttt{\$c}_\texttt{i}$
- 110 00-11 000-111 ⇒ `result` $\texttt{\$c}_\texttt{i}$ *immediate* ⇒ $\texttt{PC} = (\texttt{\$re} == immediate)\ ?\ \texttt{\$c}_\texttt{i} : \texttt{PC} + 1$

The immediate is signed and ranges from -3 to 3 (101 - 011).

Type 0R:

Operations that do not involve the use of registers.

$$\underbrace{OpCode}_{3\ bits}\ \underbrace{don't\ care}_{4\ bits}\ \underbrace{funct}_{1\ bit}$$

Control:

- 111 0000 0 ⇒ `nop` (performs no action)
- 111 0000 1 ⇒ `hlt` (ends the program)

Appendix A (additional processor features)

Project objective:

[01] - **All operations are performed in a single *clock* cycle**, with the cycle duration arbitrarily set at 10ns (since the simulation has no physical constraint, any value would be acceptable).

Architecture

[02] - `ALUOp` 01 was defined for many operations because it avoids access to multiple registers (which could end up causing *bugs* due to the control *flags*).

[03] - **The ALU generates a signal indicating when an arithmetic *overflow* occurred**. However, due to the purpose of the project and the embedded program, this signal does not affect the project's operation in any way.

Instructions (binary and Assembly)

[04] - The *don't care bits* default to 0 in the binary instructions (this avoids conflicts in the control *flags*, improper register access, etc.).

Appendix B (*testbench* results)

The *testbenches* were run on the site `https://www.edaplayground.com/` (*Testbench*/*Design*: SystemVerilog/Verilog, Compiler/Simulator: Icarus Verilog 12.0)

Individual modules:

- **Control:**

Op2	Op1	Op0	funct	reset	Sinais de controle									
0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	0	1	0	1	0	0	0	0	0	0	1
0	0	1	1	0	1	0	1	0	0	0	0	0	0	1
0	1	0	0	0	0	0	1	0	0	0	0	0	1	0
0	1	0	1	0	0	0	0	0	1	0	0	0	0	1	0
0	1	1	0	0	0	1	1	1	0	0	0	0	0	0	1
0	1	1	1	0	0	0	0	0	1	0	0	0	0	0	1
1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1
1	0	0	1	0	0	0	0	0	0	0	1	0	0	0	1
1	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1
1	0	1	1	0	0	0	0	0	0	0	0	0	0	1	1
1	1	0	0	0	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	1	0	0	0	0	0	1	0	0	0	0
1	1	1	0	0	1	1	0	0	0	0	0	1	0	0	1
1	1	1	1	0	1	0	0	0	0	0	0	0	1	0	1
1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	0	1	0	0	0	0	0	0	0	1	0	1

- ***PC*:**

Time	Clock	Reset	End	endereco	enderecoSaida
0	0	1	0	5	0
5	1	1	0	5	0
10	0	0	0	5	0
15	1	0	0	5	5
20	0	0	0	10	5
25	1	0	0	10	10
30	0	0	0	20	10
35	1	0	0	20	20
40	0	0	1	30	20
45	1	0	1	30	20
50	0	0	1	40	20
55	1	0	1	40	20
60	0	1	1	40	0
65	1	1	1	40	0
70	0	0	0	40	0
75	1	0	0	40	40
80	0	0	0	40	40

- **Data memory:**

Endereço	Dado lido
0	01001000 (H)
1	01000101 (E)
2	01001100 (L)
3	01001100 (L)
4	01001111 (O)
5	00000000 ( )
0	01001000 (H)
1	01000101 (E)
2	01001100 (L)

```
     3    | 01001100 (L)
     4    | 01001111 (O)
     5    | 00000000 ( )
     6    | 01010111 (W)
     7    | 01001111 (O)
     8    | 01010010 (R)
     9    | 01001100 (L)
    10    | 01000100 (D)
     0    | 01010111 (W)
     1    | 01001111 (O)
     2    | 01010010 (R)
     3    | 01001100 (L)
     4    | 01000100 (D)
     5    | 00000000 ( )
     6    | 01010111 (W)
     7    | 01001111 (O)
     8    | 01010010 (R)
     9    | 01001100 (L)
    10    | 01000100 (D)
    20    | xxxxxxxx ( )
    20    | 01011010 (Z)
    20    | 01011010 (Z)
```

- **Instruction memory:**

Endereço	OpCode	Reg1	Reg2	Funct	Imediato
0	000	10	10	1	101
4	001	00	01	1	011
10	011	01	10	0	100
19	110	11	00	1	001
54	001	00	01	1	011
77	111	00	00	0	000
81	100	00	00	0	000
83	111	00	00	1	001

- **Register file:**

Time	Clock	Reset	RegWrite	Reg1	Reg2	DadoW	Dado Reg1	Dado Reg2	ReIn	ReOut
0	0	1	0	00	00	00	00	00	00	fc
5	1	1	0	00	00	00	00	00	00	fc
10	0	0	1	00	00	aa	00	00	11	fc
15	1	0	1	00	00	aa	aa	aa	11	11
20	0	0	1	01	00	bb	00	aa	22	11
25	1	0	1	01	00	bb	bb	aa	22	22
30	0	0	1	10	00	cc	00	aa	33	22
35	1	0	1	10	00	cc	cc	aa	33	33
40	0	0	1	11	00	dd	00	aa	44	33
45	1	0	1	11	00	dd	dd	aa	44	44
50	0	0	0	00	01	dd	aa	bb	55	44
55	1	0	0	00	01	dd	aa	bb	55	55
60	0	0	0	10	11	dd	cc	dd	55	55
65	1	0	0	10	11	dd	cc	dd	55	55
70	0	0	0	00	00	ff	aa	aa	66	55
75	1	0	0	00	00	ff	aa	aa	66	66
80	0	0	0	00	00	ff	aa	aa	66	66
85	1	0	0	00	00	ff	aa	aa	66	66
90	0	0	1	00	00	ff	aa	aa	66	66
95	1	0	1	00	00	ff	ff	ff	66	66
100	0	0	1	00	00	ff	ff	ff	77	66
105	1	0	1	00	00	ff	ff	ff	77	77
110	0	0	1	00	00	ff	ff	ff	88	77
115	1	0	1	00	00	ff	ff	ff	88	88
120	0	1	1	00	00	ff	00	00	88	fc
125	1	1	1	00	00	ff	00	00	88	fc
130	0	0	1	00	00	ff	00	00	88	fc
135	1	0	1	00	00	ff	ff	ff	88	88
140	0	0	1	00	00	ff	ff	ff	88	88

- **ALU:**

```
  A   |  B   |  S   | overflow | reEntrada | reSaida | zero
 10   |  20  |  30  |     0    |     aa    |    aa   |  0
255   |   1  |   0  |     1    |     55    |    55   |  0
 30   |  15  |  15  |     1    |     bb    |    bb   |  0
 50   |  25  |  75  |     0    |     ee    |    ee   |  0
 10   |   5  |  50  |     0    |     ff    |    ff   |  0
128   |   2  |   0  |     1    |     66    |    66   |  0
 40   |  40  |   0  |     1    |     11    |    01   |  0
 40   |  41  |   0  |     0    |     22    |    00   |  0
 10   |  20  |   0  |     0    |     33    |    01   |  0
 30   |  20  |   0  |     1    |     44    |    00   |  0
 25   | 204  |   0  |     1    |     cc    |    fc   |  1
 30   |  25  |   0  |     0    |     dd    |    fc   |  0
```

Complete processor with embedded program:

- **Message: HELLO - Cipher: 5**

```
Memoria de dados [0:6]:
Endereco |    Valor   | Decimal | Caractere
    0    |  01001000  |    72   |     H
    1    |  01000101  |    69   |     E
    2    |  01001100  |    76   |     L
    3    |  01001100  |    76   |     L
    4    |  01001111  |    79   |     O
    5    |  00000000  |     0   |
    6    |  xxxxxxxx  |     x   |
PC | Instrução |     Assembly     |  $c0  |  $c1  |  $c2  |  $c3  |  $re
 0 |  00010101 | sub $c2, $c2     |   0   |   0   |   0   |   0   |  -4
 1 |  00000001 | sub $c0, $c0     |   0   |   0   |   0   |   0   |  -4
 2 |  00011111 | sub $c3, $c3     |   0   |   0   |   0   |   0   |  -4
 3 |  11100000 | nop              |   0   |   0   |   0   |   0   |  -4
 4 |  00100011 | addi $c0, 3      |   0   |   0   |   0   |   0   |  -4
 5 |  00100010 | addi $c0, 2      |   3   |   0   |   0   |   0   |  -4
 6 |  00100000 | addi $c0, 0      |   5   |   0   |   0   |   0   |  -4
 7 |  00100000 | addi $c0, 0      |   5   |   0   |   0   |   0   |  -4
 8 |  00100000 | addi $c0, 0      |   5   |   0   |   0   |   0   |  -4
 9 |  11100000 | nop              |   5   |   0   |   0   |   0   |  -4
10 |  01101100 | ld $c1, $c2      |   5   |   0   |   0   |   0   |  -4
11 |  00001000 | add $c1, $c0     |   5   |  72   |   0   |   0   |  -4
12 |  10101000 | beq $c1, $c0     |   5   |  77   |   0   |   0   |  -4
13 |  00000001 | sub $c0, $c0     |   5   |  77   |   0   |   0   |   0
14 |  11100000 | nop              |   0   |  77   |   0   |   0   |   0
15 |  00111011 | addi $c3, 3      |   0   |  77   |   0   |   0   |   0
16 |  01011110 | mult $c3, $c3    |   0   |  77   |   0   |   3   |   0
17 |  01011110 | mult $c3, $c3    |   0   |  77   |   0   |   9   |   0
18 |  00111001 | addi $c3, 1      |   0   |  77   |   0   |  81   |   0
19 |  11011001 | result $c3, 1    |   0   |  77   |   0   |  82   |   0
20 |  00000001 | sub $c0, $c0     |   0   |  77   |   0   |  82   |  -4
21 |  00011111 | sub $c3, $c3     |   0   |  77   |   0   |  82   |  -4
22 |  11100000 | nop              |   0   |  77   |   0   |   0   |  -4
23 |  00100011 | addi $c0, 3      |   0   |  77   |   0   |   0   |  -4
24 |  00100011 | addi $c0, 3      |   3   |  77   |   0   |   0   |  -4
25 |  00100011 | addi $c0, 3      |   6   |  77   |   0   |   0   |  -4
26 |  00100001 | addi $c0, 1      |   9   |  77   |   0   |   0   |  -4
27 |  00111011 | addi $c3, 3      |  10   |  77   |   0   |   0   |  -4
28 |  00111011 | addi $c3, 3      |  10   |  77   |   0   |   3   |  -4
29 |  00111011 | addi $c3, 3      |  10   |  77   |   0   |   6   |  -4
30 |  01000110 | mult $c0, $c3    |  10   |  77   |   0   |   9   |  -4
31 |  00100001 | addi $c0, 1      |  90   |  77   |   0   |   9   |  -4
32 |  00011111 | sub $c3, $c3     |  91   |  77   |   0   |   9   |  -4
33 |  10101001 | slt $c1, $c0     |  91   |  77   |   0   |   0   |  -4
34 |  00000001 | sub $c0, $c0     |  91   |  77   |   0   |   0   |   1
35 |  00011111 | sub $c3, $c3     |   0   |  77   |   0   |   0   |   1
36 |  00111011 | addi $c3, 3      |   0   |  77   |   0   |   0   |   1
```

```
 37 |  00000110 | add $c0, $c3     |   0   |  77   |   0   |   3   |   1
 38 |  01011000 | mult $c3, $c0    |   3   |  77   |   0   |   3   |   1
 39 |  01011000 | mult $c3, $c0    |   3   |  77   |   0   |   9   |   1
 40 |  00011110 | add $c3, $c3     |   3   |  77   |   0   |  27   |   1
 41 |  00111101 | addi $c3, 5      |   3   |  77   |   0   |  54   |   1
 42 |  11011001 | result $c3, 1    |   3   |  77   |   0   |  51   |   1
 51 |  11100000 | nop              |   3   |  77   |   0   |  51   |  -4
 52 |  00000001 | sub $c0, $c0     |   3   |  77   |   0   |  51   |  -4
 53 |  00011111 | sub $c3, $c3     |   0   |  77   |   0   |  51   |  -4
 54 |  00100011 | addi $c0, 3      |   0   |  77   |   0   |   0   |  -4
 55 |  00100001 | addi $c0, 1      |   3   |  77   |   0   |   0   |  -4
 56 |  00011000 | add $c3, $c0     |   4   |  77   |   0   |   0   |  -4
 57 |  01000110 | mult $c0, $c3    |   4   |  77   |   0   |   4   |  -4
 58 |  01000110 | mult $c0, $c3    |  16   |  77   |   0   |   4   |  -4
 59 |  00100001 | addi $c0, 1      |  64   |  77   |   0   |   4   |  -4
 60 |  00011111 | sub $c3, $c3     |  65   |  77   |   0   |   4   |  -4
 61 |  10101001 | slt $c1, $c0     |  65   |  77   |   0   |   0   |  -4
 62 |  00000001 | sub $c0, $c0     |  65   |  77   |   0   |   0   |   0
 63 |  00111011 | addi $c3, 3      |   0   |  77   |   0   |   0   |   0
 64 |  01011110 | mult $c3, $c3    |   0   |  77   |   0   |   3   |   0
 65 |  01011110 | mult $c3, $c3    |   0   |  77   |   0   |   9   |   0
 66 |  00111101 | addi $c3, 5      |   0   |  77   |   0   |  81   |   0
 67 |  11011000 | result $c3, 0    |   0   |  77   |   0   |  78   |   0
 78 |  01101101 | st $c1, $c2      |   0   |  77   |   0   |  78   |  -4
 79 |  00110001 | addi $c2, 1      |   0   |  77   |   0   |  78   |  -4
 80 |  00100001 | addi $c0, 1      |   0   |  77   |   1   |  78   |  -4
 81 |  10000000 | jr $c0           |   1   |  77   |   1   |  78   |  -4
  1 |  00000001 | sub $c0, $c0     |   1   |  77   |   1   |  78   |  -4
  2 |  00011111 | sub $c3, $c3     |   0   |  77   |   1   |  78   |  -4
  3 |  11100000 | nop              |   0   |  77   |   1   |   0   |  -4
  4 |  00100011 | addi $c0, 3      |   0   |  77   |   1   |   0   |  -4
  5 |  00100010 | addi $c0, 2      |   3   |  77   |   1   |   0   |  -4
  6 |  00100000 | addi $c0, 0      |   5   |  77   |   1   |   0   |  -4
  7 |  00100000 | addi $c0, 0      |   5   |  77   |   1   |   0   |  -4
  8 |  00100000 | addi $c0, 0      |   5   |  77   |   1   |   0   |  -4
  9 |  11100000 | nop              |   5   |  77   |   1   |   0   |  -4
 10 |  01101100 | ld $c1, $c2      |   5   |  77   |   1   |   0   |  -4
 11 |  00001000 | add $c1, $c0     |   5   |  69   |   1   |   0   |  -4
 12 |  10101000 | beq $c1, $c0     |   5   |  74   |   1   |   0   |  -4
 13 |  00000001 | sub $c0, $c0     |   5   |  74   |   1   |   0   |   0
 14 |  11100000 | nop              |   0   |  74   |   1   |   0   |   0
 15 |  00111011 | addi $c3, 3      |   0   |  74   |   1   |   0   |   0
 16 |  01011110 | mult $c3, $c3    |   0   |  74   |   1   |   3   |   0
 17 |  01011110 | mult $c3, $c3    |   0   |  74   |   1   |   9   |   0
 18 |  00111001 | addi $c3, 1      |   0   |  74   |   1   |  81   |   0
 19 |  11011001 | result $c3, 1    |   0   |  74   |   1   |  82   |   0
 20 |  00000001 | sub $c0, $c0     |   0   |  74   |   1   |  82   |  -4
 21 |  00011111 | sub $c3, $c3     |   0   |  74   |   1   |  82   |  -4
 22 |  11100000 | nop              |   0   |  74   |   1   |   0   |  -4
 23 |  00100011 | addi $c0, 3      |   0   |  74   |   1   |   0   |  -4
 24 |  00100011 | addi $c0, 3      |   3   |  74   |   1   |   0   |  -4
 25 |  00100011 | addi $c0, 3      |   6   |  74   |   1   |   0   |  -4
 26 |  00100001 | addi $c0, 1      |   9   |  74   |   1   |   0   |  -4
 27 |  00111011 | addi $c3, 3      |  10   |  74   |   1   |   0   |  -4
 28 |  00111011 | addi $c3, 3      |  10   |  74   |   1   |   3   |  -4
 29 |  00111011 | addi $c3, 3      |  10   |  74   |   1   |   6   |  -4
 30 |  01000110 | mult $c0, $c3    |  10   |  74   |   1   |   9   |  -4
 31 |  00100001 | addi $c0, 1      |  90   |  74   |   1   |   9   |  -4
 32 |  00011111 | sub $c3, $c3     |  91   |  74   |   1   |   9   |  -4
 33 |  10101001 | slt $c1, $c0     |  91   |  74   |   1   |   0   |  -4
 34 |  00000001 | sub $c0, $c0     |  91   |  74   |   1   |   0   |   1
 35 |  00011111 | sub $c3, $c3     |   0   |  74   |   1   |   0   |   1
 36 |  00111011 | addi $c3, 3      |   0   |  74   |   1   |   0   |   1
 37 |  00000110 | add $c0, $c3     |   0   |  74   |   1   |   3   |   1
 38 |  01011000 | mult $c3, $c0    |   3   |  74   |   1   |   3   |   1
 39 |  01011000 | mult $c3, $c0    |   3   |  74   |   1   |   9   |   1
 40 |  00011110 | add $c3, $c3     |   3   |  74   |   1   |  27   |   1
 41 |  00111101 | addi $c3, 5      |   3   |  74   |   1   |  54   |   1
 42 |  11011001 | result $c3, 1    |   3   |  74   |   1   |  51   |   1
 51 |  11100000 | nop              |   3   |  74   |   1   |  51   |  -4
```

```
52 |  00000001 | sub $c0, $c0    |  3  |  74  |  1  |  51  | -4
53 |  00011111 | sub $c3, $c3    |  0  |  74  |  1  |  51  | -4
54 |  00100011 | addi $c0, 3     |  0  |  74  |  1  |   0  | -4
55 |  00100001 | addi $c0, 1     |  3  |  74  |  1  |   0  | -4
56 |  00011000 | add $c3, $c0    |  4  |  74  |  1  |   0  | -4
57 |  01000110 | mult $c0, $c3   |  4  |  74  |  1  |   4  | -4
58 |  01000110 | mult $c0, $c3   | 16  |  74  |  1  |   4  | -4
59 |  00100001 | addi $c0, 1     | 64  |  74  |  1  |   4  | -4
60 |  00011111 | sub $c3, $c3    | 65  |  74  |  1  |   4  | -4
61 |  10101001 | slt $c1, $c0    | 65  |  74  |  1  |   0  | -4
62 |  00000001 | sub $c0, $c0    | 65  |  74  |  1  |   0  |  0
63 |  00111011 | addi $c3, 3     |  0  |  74  |  1  |   0  |  0
64 |  01011110 | mult $c3, $c3   |  0  |  74  |  1  |   3  |  0
65 |  01011110 | mult $c3, $c3   |  0  |  74  |  1  |   9  |  0
66 |  00111101 | addi $c3, 5     |  0  |  74  |  1  |  81  |  0
67 |  11011000 | result $c3, 0   |  0  |  74  |  1  |  78  |  0
78 |  01101101 | st $c1, $c2     |  0  |  74  |  1  |  78  | -4
79 |  00110001 | addi $c2, 1     |  0  |  74  |  1  |  78  | -4
80 |  00100001 | addi $c0, 1     |  0  |  74  |  2  |  78  | -4
81 |  10000000 | jr $c0          |  1  |  74  |  2  |  78  | -4
 1 |  00000001 | sub $c0, $c0    |  1  |  74  |  2  |  78  | -4
 2 |  00011111 | sub $c3, $c3    |  0  |  74  |  2  |  78  | -4
 3 |  11100000 | nop             |  0  |  74  |  2  |   0  | -4
 4 |  00100011 | addi $c0, 3     |  0  |  74  |  2  |   0  | -4
 5 |  00100010 | addi $c0, 2     |  3  |  74  |  2  |   0  | -4
 6 |  00100000 | addi $c0, 0     |  5  |  74  |  2  |   0  | -4
 7 |  00100000 | addi $c0, 0     |  5  |  74  |  2  |   0  | -4
 8 |  00100000 | addi $c0, 0     |  5  |  74  |  2  |   0  | -4
 9 |  11100000 | nop             |  5  |  74  |  2  |   0  | -4
10 |  01101100 | ld $c1, $c2     |  5  |  74  |  2  |   0  | -4
11 |  00001000 | add $c1, $c0    |  5  |  76  |  2  |   0  | -4
12 |  10101000 | beq $c1, $c0    |  5  |  81  |  2  |   0  | -4
13 |  00000001 | sub $c0, $c0    |  5  |  81  |  2  |   0  |  0
14 |  11100000 | nop             |  0  |  81  |  2  |   0  |  0
15 |  00111011 | addi $c3, 3     |  0  |  81  |  2  |   0  |  0
16 |  01011110 | mult $c3, $c3   |  0  |  81  |  2  |   3  |  0
17 |  01011110 | mult $c3, $c3   |  0  |  81  |  2  |   9  |  0
18 |  00111001 | addi $c3, 1     |  0  |  81  |  2  |  81  |  0
19 |  11011001 | result $c3, 1   |  0  |  81  |  2  |  82  |  0
20 |  00000001 | sub $c0, $c0    |  0  |  81  |  2  |  82  | -4
21 |  00011111 | sub $c3, $c3    |  0  |  81  |  2  |  82  | -4
22 |  11100000 | nop             |  0  |  81  |  2  |   0  | -4
23 |  00100011 | addi $c0, 3     |  0  |  81  |  2  |   0  | -4
24 |  00100011 | addi $c0, 3     |  3  |  81  |  2  |   0  | -4
25 |  00100011 | addi $c0, 3     |  6  |  81  |  2  |   0  | -4
26 |  00100001 | addi $c0, 1     |  9  |  81  |  2  |   0  | -4
27 |  00111011 | addi $c3, 3     | 10  |  81  |  2  |   0  | -4
28 |  00111011 | addi $c3, 3     | 10  |  81  |  2  |   3  | -4
29 |  00111011 | addi $c3, 3     | 10  |  81  |  2  |   6  | -4
30 |  01000110 | mult $c0, $c3   | 10  |  81  |  2  |   9  | -4
31 |  00100001 | addi $c0, 1     | 90  |  81  |  2  |   9  | -4
32 |  00011111 | sub $c3, $c3    | 91  |  81  |  2  |   9  | -4
33 |  10101001 | slt $c1, $c0    | 91  |  81  |  2  |   0  | -4
34 |  00000001 | sub $c0, $c0    | 91  |  81  |  2  |   0  |  1
35 |  00011111 | sub $c3, $c3    |  0  |  81  |  2  |   0  |  1
36 |  00111011 | addi $c3, 3     |  0  |  81  |  2  |   0  |  1
37 |  00000110 | add $c0, $c3    |  0  |  81  |  2  |   3  |  1
38 |  01011000 | mult $c3, $c0   |  3  |  81  |  2  |   3  |  1
39 |  01011000 | mult $c3, $c0   |  3  |  81  |  2  |   9  |  1
40 |  00011110 | add $c3, $c3    |  3  |  81  |  2  |  27  |  1
41 |  00111101 | addi $c3, 5     |  3  |  81  |  2  |  54  |  1
42 |  11011001 | result $c3, 1   |  3  |  81  |  2  |  51  |  1
51 |  11100000 | nop             |  3  |  81  |  2  |  51  | -4
52 |  00000001 | sub $c0, $c0    |  3  |  81  |  2  |  51  | -4
53 |  00011111 | sub $c3, $c3    |  0  |  81  |  2  |  51  | -4
54 |  00100011 | addi $c0, 3     |  0  |  81  |  2  |   0  | -4
55 |  00100001 | addi $c0, 1     |  3  |  81  |  2  |   0  | -4
56 |  00011000 | add $c3, $c0    |  4  |  81  |  2  |   0  | -4
57 |  01000110 | mult $c0, $c3   |  4  |  81  |  2  |   4  | -4
58 |  01000110 | mult $c0, $c3   | 16  |  81  |  2  |   4  | -4
```

```
 59 |  00100001 | addi $c0, 1     |  64  |  81  |  2  |  4  | -4
 60 |  00011111 | sub $c3, $c3    |  65  |  81  |  2  |  4  | -4
 61 |  10101001 | slt $c1, $c0    |  65  |  81  |  2  |  0  | -4
 62 |  00000001 | sub $c0, $c0    |  65  |  81  |  2  |  0  |  0
 63 |  00111011 | addi $c3, 3     |   0  |  81  |  2  |  0  |  0
 64 |  01011110 | mult $c3, $c3   |   0  |  81  |  2  |  3  |  0
 65 |  01011110 | mult $c3, $c3   |   0  |  81  |  2  |  9  |  0
 66 |  00111101 | addi $c3, 5     |   0  |  81  |  2  | 81  |  0
 67 |  11011000 | result $c3, 0   |   0  |  81  |  2  | 78  |  0
 78 |  01101101 | st $c1, $c2     |   0  |  81  |  2  | 78  | -4
 79 |  00110001 | addi $c2, 1     |   0  |  81  |  2  | 78  | -4
 80 |  00100001 | addi $c0, 1     |   0  |  81  |  3  | 78  | -4
 81 |  10000000 | jr $c0          |   1  |  81  |  3  | 78  | -4
  1 |  00000001 | sub $c0, $c0    |   1  |  81  |  3  | 78  | -4
  2 |  00011111 | sub $c3, $c3    |   0  |  81  |  3  | 78  | -4
  3 |  11100000 | nop             |   0  |  81  |  3  |  0  | -4
  4 |  00100011 | addi $c0, 3     |   0  |  81  |  3  |  0  | -4
  5 |  00100010 | addi $c0, 2     |   3  |  81  |  3  |  0  | -4
  6 |  00100000 | addi $c0, 0     |   5  |  81  |  3  |  0  | -4
  7 |  00100000 | addi $c0, 0     |   5  |  81  |  3  |  0  | -4
  8 |  00100000 | addi $c0, 0     |   5  |  81  |  3  |  0  | -4
  9 |  11100000 | nop             |   5  |  81  |  3  |  0  | -4
 10 |  01101100 | ld $c1, $c2     |   5  |  81  |  3  |  0  | -4
 11 |  00001000 | add $c1, $c0    |   5  |  76  |  3  |  0  | -4
 12 |  10101000 | beq $c1, $c0    |   5  |  81  |  3  |  0  | -4
 13 |  00000001 | sub $c0, $c0    |   5  |  81  |  3  |  0  |  0
 14 |  11100000 | nop             |   0  |  81  |  3  |  0  |  0
 15 |  00111011 | addi $c3, 3     |   0  |  81  |  3  |  0  |  0
 16 |  01011110 | mult $c3, $c3   |   0  |  81  |  3  |  3  |  0
 17 |  01011110 | mult $c3, $c3   |   0  |  81  |  3  |  9  |  0
 18 |  00111001 | addi $c3, 1     |   0  |  81  |  3  | 81  |  0
 19 |  11011001 | result $c3, 1   |   0  |  81  |  3  | 82  |  0
 20 |  00000001 | sub $c0, $c0    |   0  |  81  |  3  | 82  | -4
 21 |  00011111 | sub $c3, $c3    |   0  |  81  |  3  | 82  | -4
 22 |  11100000 | nop             |   0  |  81  |  3  |  0  | -4
 23 |  00100011 | addi $c0, 3     |   0  |  81  |  3  |  0  | -4
 24 |  00100011 | addi $c0, 3     |   3  |  81  |  3  |  0  | -4
 25 |  00100011 | addi $c0, 3     |   6  |  81  |  3  |  0  | -4
 26 |  00100001 | addi $c0, 1     |   9  |  81  |  3  |  0  | -4
 27 |  00111011 | addi $c3, 3     |  10  |  81  |  3  |  0  | -4
 28 |  00111011 | addi $c3, 3     |  10  |  81  |  3  |  3  | -4
 29 |  00111011 | addi $c3, 3     |  10  |  81  |  3  |  6  | -4
 30 |  01000110 | mult $c0, $c3   |  10  |  81  |  3  |  9  | -4
 31 |  00100001 | addi $c0, 1     |  90  |  81  |  3  |  9  | -4
 32 |  00011111 | sub $c3, $c3    |  91  |  81  |  3  |  9  | -4
 33 |  10101001 | slt $c1, $c0    |  91  |  81  |  3  |  0  | -4
 34 |  00000001 | sub $c0, $c0    |  91  |  81  |  3  |  0  |  1
 35 |  00011111 | sub $c3, $c3    |   0  |  81  |  3  |  0  |  1
 36 |  00111011 | addi $c3, 3     |   0  |  81  |  3  |  0  |  1
 37 |  00000110 | add $c0, $c3    |   0  |  81  |  3  |  3  |  1
 38 |  01011000 | mult $c3, $c0   |   3  |  81  |  3  |  3  |  1
 39 |  01011000 | mult $c3, $c0   |   3  |  81  |  3  |  9  |  1
 40 |  00011110 | add $c3, $c3    |   3  |  81  |  3  | 27  |  1
 41 |  00111101 | addi $c3, 5     |   3  |  81  |  3  | 54  |  1
 42 |  11011001 | result $c3, 1   |   3  |  81  |  3  | 51  |  1
 51 |  11100000 | nop             |   3  |  81  |  3  | 51  | -4
 52 |  00000001 | sub $c0, $c0    |   3  |  81  |  3  | 51  | -4
 53 |  00011111 | sub $c3, $c3    |   0  |  81  |  3  | 51  | -4
 54 |  00100011 | addi $c0, 3     |   0  |  81  |  3  |  0  | -4
 55 |  00100001 | addi $c0, 1     |   3  |  81  |  3  |  0  | -4
 56 |  00011000 | add $c3, $c0    |   4  |  81  |  3  |  0  | -4
 57 |  01000110 | mult $c0, $c3   |   4  |  81  |  3  |  4  | -4
 58 |  01000110 | mult $c0, $c3   |  16  |  81  |  3  |  4  | -4
 59 |  00100001 | addi $c0, 1     |  64  |  81  |  3  |  4  | -4
 60 |  00011111 | sub $c3, $c3    |  65  |  81  |  3  |  4  | -4
 61 |  10101001 | slt $c1, $c0    |  65  |  81  |  3  |  0  | -4
 62 |  00000001 | sub $c0, $c0    |  65  |  81  |  3  |  0  |  0
 63 |  00111011 | addi $c3, 3     |   0  |  81  |  3  |  0  |  0
 64 |  01011110 | mult $c3, $c3   |   0  |  81  |  3  |  3  |  0
 65 |  01011110 | mult $c3, $c3   |   0  |  81  |  3  |  9  |  0
```

```
 66 |  00111101 | addi $c3, 5     |   0   |  81   |   3   |  81   |   0
 67 |  11011000 | result $c3, 0   |   0   |  81   |   3   |  78   |   0
 78 |  01101101 | st $c1, $c2     |   0   |  81   |   3   |  78   |  -4
 79 |  00110001 | addi $c2, 1     |   0   |  81   |   3   |  78   |  -4
 80 |  00100001 | addi $c0, 1     |   0   |  81   |   4   |  78   |  -4
 81 |  10000000 | jr $c0          |   1   |  81   |   4   |  78   |  -4
  1 |  00000001 | sub $c0, $c0    |   1   |  81   |   4   |  78   |  -4
  2 |  00011111 | sub $c3, $c3    |   0   |  81   |   4   |  78   |  -4
  3 |  11100000 | nop             |   0   |  81   |   4   |   0   |  -4
  4 |  00100011 | addi $c0, 3     |   0   |  81   |   4   |   0   |  -4
  5 |  00100010 | addi $c0, 2     |   3   |  81   |   4   |   0   |  -4
  6 |  00100000 | addi $c0, 0     |   5   |  81   |   4   |   0   |  -4
  7 |  00100000 | addi $c0, 0     |   5   |  81   |   4   |   0   |  -4
  8 |  00100000 | addi $c0, 0     |   5   |  81   |   4   |   0   |  -4
  9 |  11100000 | nop             |   5   |  81   |   4   |   0   |  -4
 10 |  01101100 | ld $c1, $c2     |   5   |  81   |   4   |   0   |  -4
 11 |  00001000 | add $c1, $c0    |   5   |  79   |   4   |   0   |  -4
 12 |  10101000 | beq $c1, $c0    |   5   |  84   |   4   |   0   |  -4
 13 |  00000001 | sub $c0, $c0    |   5   |  84   |   4   |   0   |   0
 14 |  11100000 | nop             |   0   |  84   |   4   |   0   |   0
 15 |  00111011 | addi $c3, 3     |   0   |  84   |   4   |   0   |   0
 16 |  01011110 | mult $c3, $c3   |   0   |  84   |   4   |   3   |   0
 17 |  01011110 | mult $c3, $c3   |   0   |  84   |   4   |   9   |   0
 18 |  00111001 | addi $c3, 1     |   0   |  84   |   4   |  81   |   0
 19 |  11011001 | result $c3, 1   |   0   |  84   |   4   |  82   |   0
 20 |  00000001 | sub $c0, $c0    |   0   |  84   |   4   |  82   |  -4
 21 |  00011111 | sub $c3, $c3    |   0   |  84   |   4   |  82   |  -4
 22 |  11100000 | nop             |   0   |  84   |   4   |   0   |  -4
 23 |  00100011 | addi $c0, 3     |   0   |  84   |   4   |   0   |  -4
 24 |  00100011 | addi $c0, 3     |   3   |  84   |   4   |   0   |  -4
 25 |  00100011 | addi $c0, 3     |   6   |  84   |   4   |   0   |  -4
 26 |  00100001 | addi $c0, 1     |   9   |  84   |   4   |   0   |  -4
 27 |  00111011 | addi $c3, 3     |  10   |  84   |   4   |   0   |  -4
 28 |  00111011 | addi $c3, 3     |  10   |  84   |   4   |   3   |  -4
 29 |  00111011 | addi $c3, 3     |  10   |  84   |   4   |   6   |  -4
 30 |  01000110 | mult $c0, $c3   |  10   |  84   |   4   |   9   |  -4
 31 |  00100001 | addi $c0, 1     |  90   |  84   |   4   |   9   |  -4
 32 |  00011111 | sub $c3, $c3    |  91   |  84   |   4   |   9   |  -4
 33 |  10101001 | slt $c1, $c0    |  91   |  84   |   4   |   0   |  -4
 34 |  00000001 | sub $c0, $c0    |  91   |  84   |   4   |   0   |   1
 35 |  00011111 | sub $c3, $c3    |   0   |  84   |   4   |   0   |   1
 36 |  00111011 | addi $c3, 3     |   0   |  84   |   4   |   0   |   1
 37 |  00000110 | add $c0, $c3    |   0   |  84   |   4   |   3   |   1
 38 |  01011000 | mult $c3, $c0   |   3   |  84   |   4   |   3   |   1
 39 |  01011000 | mult $c3, $c0   |   3   |  84   |   4   |   9   |   1
 40 |  00011110 | add $c3, $c3    |   3   |  84   |   4   |  27   |   1
 41 |  00111101 | addi $c3, 5     |   3   |  84   |   4   |  54   |   1
 42 |  11011001 | result $c3, 1   |   3   |  84   |   4   |  51   |   1
 51 |  11100000 | nop             |   3   |  84   |   4   |  51   |  -4
 52 |  00000001 | sub $c0, $c0    |   3   |  84   |   4   |  51   |  -4
 53 |  00011111 | sub $c3, $c3    |   0   |  84   |   4   |  51   |  -4
 54 |  00100011 | addi $c0, 3     |   0   |  84   |   4   |   0   |  -4
 55 |  00100001 | addi $c0, 1     |   3   |  84   |   4   |   0   |  -4
 56 |  00011000 | add $c3, $c0    |   4   |  84   |   4   |   0   |  -4
 57 |  01000110 | mult $c0, $c3   |   4   |  84   |   4   |   4   |  -4
 58 |  01000110 | mult $c0, $c3   |  16   |  84   |   4   |   4   |  -4
 59 |  00100001 | addi $c0, 1     |  64   |  84   |   4   |   4   |  -4
 60 |  00011111 | sub $c3, $c3    |  65   |  84   |   4   |   4   |  -4
 61 |  10101001 | slt $c1, $c0    |  65   |  84   |   4   |   0   |  -4
 62 |  00000001 | sub $c0, $c0    |  65   |  84   |   4   |   0   |   0
 63 |  00111011 | addi $c3, 3     |   0   |  84   |   4   |   0   |   0
 64 |  01011110 | mult $c3, $c3   |   0   |  84   |   4   |   3   |   0
 65 |  01011110 | mult $c3, $c3   |   0   |  84   |   4   |   9   |   0
 66 |  00111101 | addi $c3, 5     |   0   |  84   |   4   |  81   |   0
 67 |  11011000 | result $c3, 0   |   0   |  84   |   4   |  78   |   0
 78 |  01101101 | st $c1, $c2     |   0   |  84   |   4   |  78   |  -4
 79 |  00110001 | addi $c2, 1     |   0   |  84   |   4   |  78   |  -4
 80 |  00100001 | addi $c0, 1     |   0   |  84   |   5   |  78   |  -4
 81 |  10000000 | jr $c0          |   1   |  84   |   5   |  78   |  -4
  1 |  00000001 | sub $c0, $c0    |   1   |  84   |   5   |  78   |  -4
```

```
 2 |  00011111 | sub $c3, $c3     |   0   |  84   |   5   |  78   |  -4
 3 |  11100000 | nop              |   0   |  84   |   5   |   0   |  -4
 4 |  00100011 | addi $c0, 3      |   0   |  84   |   5   |   0   |  -4
 5 |  00100010 | addi $c0, 2      |   3   |  84   |   5   |   0   |  -4
 6 |  00100000 | addi $c0, 0      |   5   |  84   |   5   |   0   |  -4
 7 |  00100000 | addi $c0, 0      |   5   |  84   |   5   |   0   |  -4
 8 |  00100000 | addi $c0, 0      |   5   |  84   |   5   |   0   |  -4
 9 |  11100000 | nop              |   5   |  84   |   5   |   0   |  -4
10 |  01101100 | ld $c1, $c2      |   5   |  84   |   5   |   0   |  -4
11 |  00001000 | add $c1, $c0     |   5   |   0   |   5   |   0   |  -4
12 |  10101000 | beq $c1, $c0     |   5   |   5   |   5   |   0   |  -4
13 |  00000001 | sub $c0, $c0     |   5   |   5   |   5   |   0   |   1
14 |  11100000 | nop              |   0   |   5   |   5   |   0   |   1
15 |  00111011 | addi $c3, 3      |   0   |   5   |   5   |   0   |   1
16 |  01011110 | mult $c3, $c3    |   0   |   5   |   5   |   3   |   1
17 |  01011110 | mult $c3, $c3    |   0   |   5   |   5   |   9   |   1
18 |  00111001 | addi $c3, 1      |   0   |   5   |   5   |  81   |   1
19 |  11011001 | result $c3, 1    |   0   |   5   |   5   |  82   |   1
82 |  11100000 | nop              |   0   |   5   |   5   |  82   |  -4
83 |  11100001 | hlt              |   0   |   5   |   5   |  82   |  -4
Memoria de dados [0:6]:
Endereco |    Valor   | Decimal | Caractere
    0    |  01001101  |    77   |     M
    1    |  01001010  |    74   |     J
    2    |  01010001  |    81   |     Q
    3    |  01010001  |    81   |     Q
    4    |  01010100  |    84   |     T
    5    |  00000000  |     0   |
    6    |  xxxxxxxx  |     x   |
```

- **summarized *testbench* (Message: BATATA - Cipher: 15 -** `show_data_memory(0, 7);`**)**

```
Memoria de dados [0:7]:
Endereco |    Valor   | Decimal | Caractere
    0    |  01000010  |    66   |     B
    1    |  01000001  |    65   |     A
    2    |  01010100  |    84   |     T
    3    |  01000001  |    65   |     A
    4    |  01010100  |    84   |     T
    5    |  01000001  |    65   |     A
    6    |  00000000  |     0   |
    7    |  xxxxxxxx  |     x   |
Memoria de dados [0:7]:
Endereco |    Valor   | Decimal | Caractere
    0    |  01010001  |    81   |     Q
    1    |  01010000  |    80   |     P
    2    |  01001001  |    73   |     I
    3    |  01010000  |    80   |     P
    4    |  01001001  |    73   |     I
    5    |  01010000  |    80   |     P
    6    |  00000000  |     0   |
    7    |  xxxxxxxx  |     x   |
```

- **summarized *testbench* (Message: GALOFRITO - Cipher: -15 -** `show_data_memory(0, 10);`**)**

```
Memoria de dados [0:10]:
Endereco |    Valor   | Decimal | Caractere
    0    |  01000111  |    71   |     G
    1    |  01000001  |    65   |     A
    2    |  01001100  |    76   |     L
    3    |  01001111  |    79   |     O
    4    |  01000110  |    70   |     F
    5    |  01010010  |    82   |     R
    6    |  01001001  |    73   |     I
    7    |  01010100  |    84   |     T
    8    |  01001111  |    79   |     O
```

```
     9    |  00000000  |     0   |
    10    |  xxxxxxxx  |     x   |
Memoria de dados [0:10]:
Endereco |    Valor   | Decimal | Caractere
    0    |  01010010  |    82   |     R
    1    |  01001100  |    76   |     L
    2    |  01010111  |    87   |     W
    3    |  01011010  |    90   |     Z
    4    |  01010001  |    81   |     Q
    5    |  01000011  |    67   |     C
    6    |  01010100  |    84   |     T
    7    |  01000101  |    69   |     E
    8    |  01011010  |    90   |     Z
    9    |  00000000  |     0   |
   10    |  xxxxxxxx  |     x   |
```

Cipher with elements past *EOF*:

- **summarized *testbench* (Message: GALO FRITO - Cipher: -15 -** `show_data_memory(0, 11);`**)**

```
Memoria de dados [0:11]:
Endereco |    Valor   | Decimal | Caractere
    0    |  01000111  |    71   |     G
    1    |  01000001  |    65   |     A
    2    |  01001100  |    76   |     L
    3    |  01001111  |    79   |     O
    4    |  00000000  |     0   |
    5    |  01000110  |    70   |     F
    6    |  01010010  |    82   |     R
    7    |  01001001  |    73   |     I
    8    |  01010100  |    84   |     T
    9    |  01001111  |    79   |     O
   10    |  00000000  |     0   |
   11    |  xxxxxxxx  |     x   |
Memoria de dados [0:11]:
Endereco |    Valor   | Decimal | Caractere
    0    |  01010010  |    82   |     R
    1    |  01001100  |    76   |     L
    2    |  01010111  |    87   |     W
    3    |  01011010  |    90   |     Z
    4    |  00000000  |     0   |
    5    |  01000110  |    70   |     F
    6    |  01010010  |    82   |     R
    7    |  01001001  |    73   |     I
    8    |  01010100  |    84   |     T
    9    |  01001111  |    79   |     O
   10    |  00000000  |     0   |
   11    |  xxxxxxxx  |     x   |
```

Appendix C (embedded program, binary and Assembly)

```
// To make the code programmable, without a cipher change altering the number of lines (since additions with an immediate can only be done in increments of 3, and jumps target a specific line), a sum of 5 lines is always performed (ranging from -15 to 15, which covers the entire range)
 0- 00010101  // sub $c2, $c2 (zeroes the registers)
 1- 00000001  // sub $c0, $c0
 2- 00011111  // sub $c3, $c3
 3- 11100000  // nop
 4- 00100011  // addi $c0, 3 (increments the cipher value)
 5- 00100010  // addi $c0, 2 (since the remainder is 0, the cipher is 5)
 6- 00100000  // addi $c0, 0
 7- 00100000  // addi $c0, 0
 8- 00100000  // addi $c0, 0
 9- 11100000  // nop
10- 01101100  // ld $c1, $c2 (reads the value stored at the position pointed to by $c2 and copies it into $c1)
11- 00001000  // add $c1, $c0 (adds the cipher to the value)
12- 10101000  // beq $c1, $c0 (tests whether EOF (0) has been reached)
13- 00000001  // sub $c0, $c0
14- 11100000  // nop
15- 00111011  // addi $c3, 3
16- 01011110  // mult $c3, $c3
17- 01011110  // mult $c3, $c3
18- 00111001  // addi $c3, 1 // ($c3 == 82)
19- 11011001  // result $c3, 1 (if EOF has been reached, jumps to the end of the code)
20- 00000001  // sub $c0, $c0
21- 00011111  // sub $c3, $c3
22- 11100000  // nop
23- 00100011  // addi $c0, 3
24- 00100011  // addi $c0, 3
25- 00100011  // addi $c0, 3
26- 00100001  // addi $c0, 1
27- 00111011  // addi $c3, 3
28- 00111011  // addi $c3, 3
29- 00111011  // addi $c3, 3
30- 01000110  // mult $c0, $c3
31- 00100001  // addi $c0, 1 ($c0 == 91)
32- 00011111  // sub $c3, $c3
33- 10101001  // slt $c1, $c0 (checks whether $c1 < 91 (91 = 'Z') ? $re = 1 : $re = 0)
34- 00000001  // sub $c0, $c0
35- 00011111  // sub $c3, $c3
36- 00111011  // addi $c3, 3
37- 00000110  // add $c0, $c3
38- 01011000  // mult $c3, $c0
39- 01011000  // mult $c3, $c0
40- 00011110  // add $c3, $c3
41- 00111101  // addi $c3, -3 ($c3 == 51)
42- 11011001  // result $c3, 1  (if the letter is within range, skips the correction operation)
43- 00011111  // sub $c3, $c3
44- 00000001  // sub $c0, $c0
45- 00100011  // addi $c0, 3
46- 00111011  // addi $c3, 3
47- 01000110  // mult $c0, $c3
48- 01000110  // mult $c0, $c3
49- 00100111  // addi $c0, -1 ($c0 == 26)
50- 00001001  // sub $c1, $c0 (the value is decremented to fit within the range)
51- 11100000  // nop
52- 00000001  // sub $c0, $c0
53- 00011111  // sub $c3, $c3
54- 00100011  // addi $c0, 3
55- 00100001  // addi $c0, 1
56- 00011000  // add $c3, $c0
57- 01000110  // mult $c0, $c3
58- 01000110  // mult $c0, $c3
59- 00100001  // addi $c0, 1 ($c0 == 65)
60- 00011111  // sub $c3, $c3
61- 10101001  // slt $c1, $c0 (checks whether $c1 < 65 (65 = 'A') ? $re = 1 : $re = 0)
62- 00000001  // sub $c0, $c0
63- 00111011  // addi $c3, 3
64- 01011110  // mult $c3, $c3
```

```
65- 01011110  // mult $c3, $c3
66- 00111101  // addi $c3, -3 (c3 == 78)
67- 11011000  // result $c3, 0 (if the letter is within range, skips the correction operation)
68- 00011111  // sub $c3, $c3
69- 00000001  // sub $c0, $c0
70- 00100011  // addi $c0, 3
71- 00111011  // addi $c3, 3
72- 01000110  // mult $c0, $c3
73- 01000110  // mult $c0, $c3
74- 00100111  // addi $c0, -1 ($c0 == 26)
75- 00001000  // add $c1, $c0 (the value is incremented to fit within the range)
76- 00000001  // sub $c0, $c0
77- 11100000  // nop
78- 01101101  // st $c1, $c2 (stores the value of $c1 at the address pointed to by $c2)
79- 00110001  // addi $c2, 1 (moves to the next character)
80- 00100001  // addi $c0, 1
81- 10000000  // jr $c0 (returns to the beginning of the code)
82- 11100000  // nop
83- 11100001  // hlt (ends the program)
```